

COMPUTER ARCHITECTURE LAB-02

Single Bus Data-path

Name: Tanishka Neeraj Trivedi

Roll no. : B24EE1080

1. AIM: -

To design and implement a 32-bit single-bus datapath using Verilog HDL and verify its functionality through simulation by executing the specified micro-operations using register transfers and ALU operations.

2. OBJECTIVES: -

The primary objectives of this experiment are:

- To design a single-bus datapath consisting of 32-bit registers, a common bus, a multiplexer, and an Arithmetic Logic Unit (ALU).
- To implement controlled data transfers between registers using appropriate input and output control signals.
- To design an ALU capable of performing ADD, SUB, AND, and XOR operations.
- To implement ALU operations using intermediate Y and Z registers, ensuring that data transfers take place through the common bus.
- To execute the specified micro-operations and verify their correctness using a Verilog testbench.
- To analyze the simulation waveforms and confirm the proper functioning of the datapath.

3. THEORY: -

A. Single Data-path Bus

A single-bus datapath is a processor datapath architecture in which multiple registers and functional units communicate through a common 32-bit bus. The bus provides a shared path for transferring data between registers, reducing the amount

of hardware required compared to a multiple-bus architecture. Since only one source can drive the bus at a time, appropriate control signals are used to select the source register and destination register for every transfer.

The major components are as follows:

- *ALU*- It performs the Arithmetic and Logic operations
- *Program Counter (PC)*- Stores the address of the next instruction
- *Memory Address Register (MAR)*- Holds the memory address for a memory operation
- *Memory Data Register (MDR)*- Holds data transferred to or from memory
- *IR (Instruction Register)* - Stores the current instruction
- *R0-R3*- General-purpose registers used for data storage and operations
- *Y Register* - Temporarily stores one operand for ALU operations
- *Z Register*- Stores the intermediate result generated by the ALU
- *TEMP Register*- Provides temporary data storage
- *32-bit Common Bus*- Transfers data between registers and functional units
- *Multiplexer (MUX)*- Selects either the Y register output or the constant 4 as the ALU A input

All registers are 32 bits wide and operate synchronously with the clock. A reset signal initializes the registers to a known state.

B. Common-Bus operation

The common bus facilitates data transfer between registers.

Each register is provided with appropriate input and output control signals:

- **Output enable:** Allows the register to place its contents on the common bus.
- **Input enable:** Allows the register to load data from the common bus at the active clock edge.

For example, the micro-operation

$$R2 \leftarrow R1$$

is performed by enabling the output of R1 and the input of R2.

The data present in R1 is placed on the common bus and loaded into R2 at the next active clock edge. Only one register should drive the bus at a time to avoid bus contention.

C. ALU

The ALU performs arithmetic and logical operations on two 32-bit operands. The Y register stores an intermediate operand, while the Z register captures the ALU output. The result is subsequently transferred to the destination register through the common bus. This arrangement ensures that ALU operations are performed using the intermediate registers and common bus, as required by the datapath architecture.

D. Data-path Architecture

The implemented datapath consists of a 32-bit common bus connecting the PC, MAR, MDR, IR, general-purpose registers R0–R3, Y, Z, and TEMP. The ALU receives its A input from a multiplexer that selects either the contents of Y or the constant 4. Its B input is connected to the common bus. The ALU output is stored in the Z register. The Z register can then place the computed result on the common bus, allowing it to be transferred to the required destination register. The datapath uses control signals to manage register input, register output, ALU operation selection, and multiplexer selection.

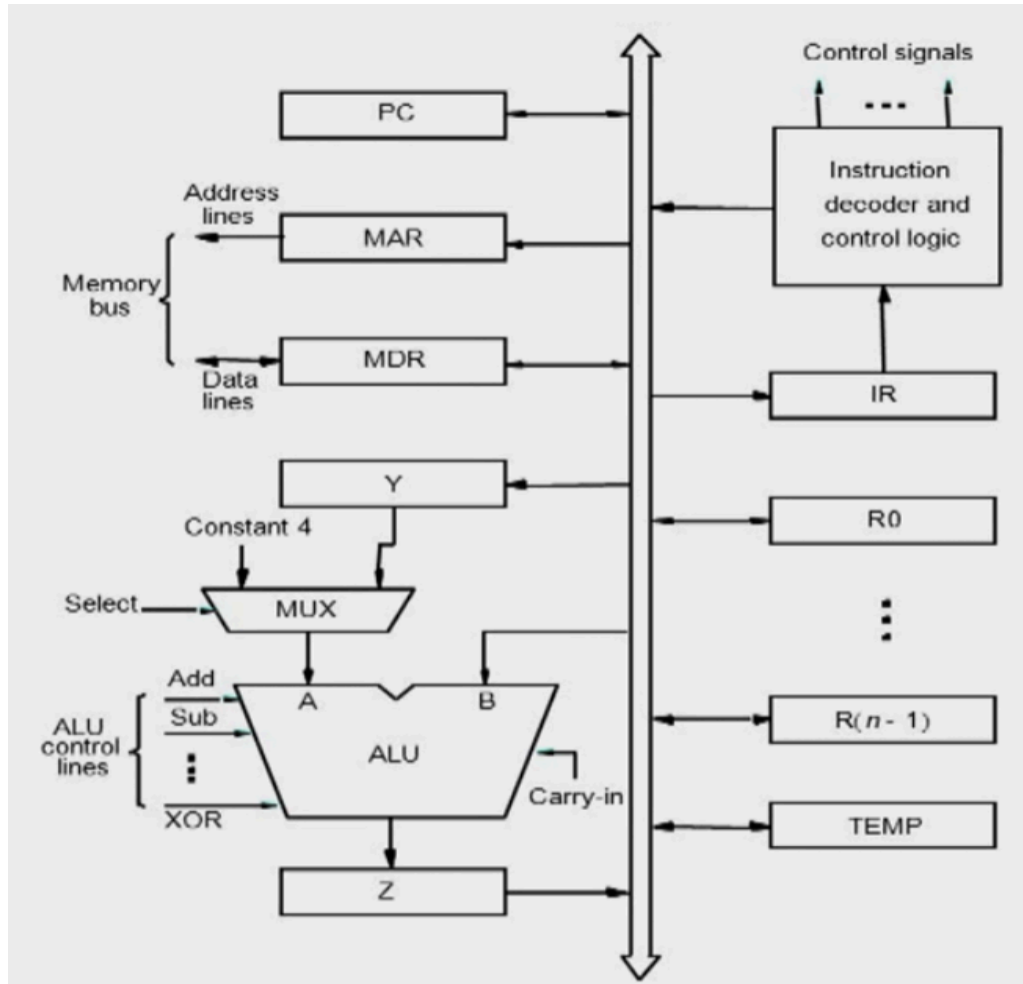


Figure.1- Block Diagram of the Single-bus data path implemented

4. MICRO-OPERATION SEQUENCES: -

a. Micro-operation 1: $R2 \leftarrow R1$

The contents of R1 are transferred to R2 through the common bus. R1 places its contents on the bus, and R2 loads the value

b. Micro-operation 2: $R3 \leftarrow R1 + R2$

Load R1 into Y through the bus. Add Y and R2, and store the result in Z
 .Transfer the result from Z to R3 through the bus .

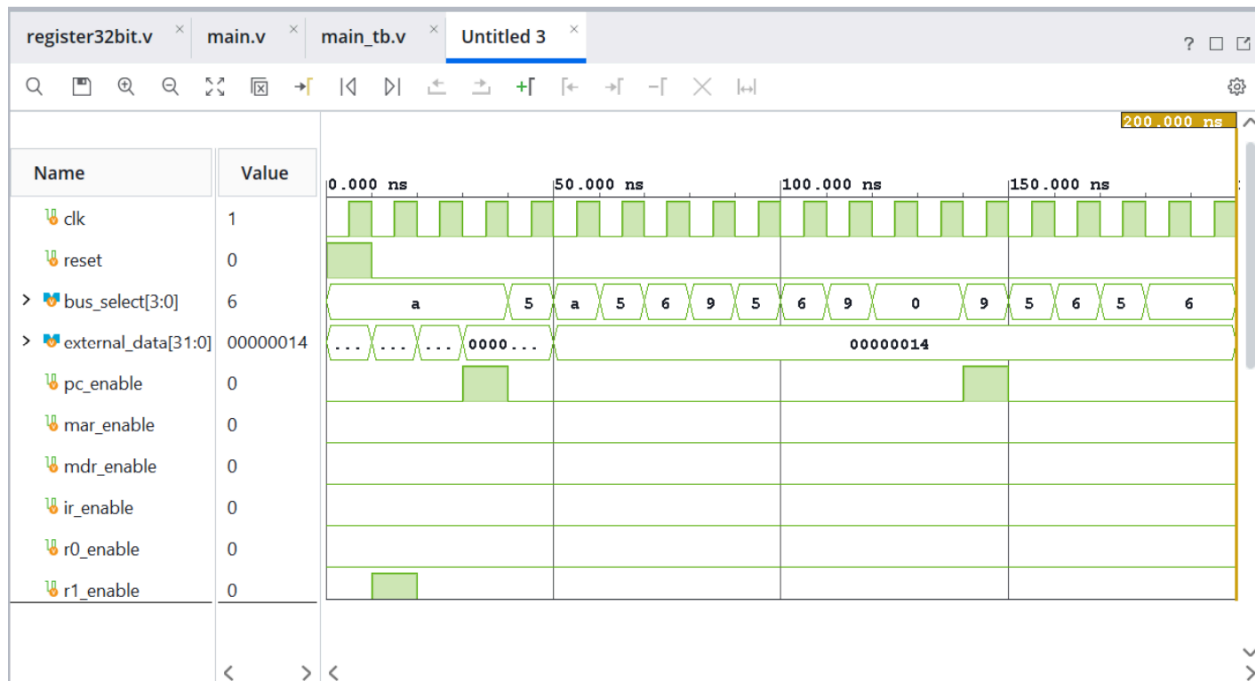
c. Micro-operation 3: $R3 \leftarrow R1 - R2$

Load R1 into Y through the bus. Subtract R2 from Y and store the result in Z. Transfer the result from Z to R3 through the bus.

d. Micro-operation 4: $PC \leftarrow PC + 4$

Load the PC value into Y through the bus. Add the constant 4 to Y and store the result in Z. Transfer the result from Z to PC through the bus.

5. SIMULATION & SOURCE CODE: -



GITHUB REPO:

<https://github.com/tanishka-trivedi/Computer-Architecture-LAB02/tree/main>

6. CONCLUSION: -

The 32-bit single-bus datapath was successfully designed and implemented using Verilog HDL. The design incorporated the required registers, a common bus, a multiplexer, and an ALU capable of performing arithmetic and logical operations.

The specified micro-operations were implemented using appropriate register control signals and intermediate Y and Z registers. The ALU results were transferred through the common bus rather than being directly assigned to the destination registers.

A Verilog testbench was developed to initialize the registers, execute the required micro-operations, and verify their functionality through simulation waveforms.

The experiment provided an understanding of single-bus datapath architecture, register transfer operations, ALU-based computation, and control signal sequencing in a processor datapath.